

ANALYSTLAB AFRICA

Machine Learning Internship Programme

Week 2 – Data Preprocessing & Feature Engineering for Machine Learning

Data Preprocessing Report

Customer Churn Prediction – ABC Communications Ltd

Prepared by: Rudolf
Role: Junior Machine Learning Engineer
Programme: Machine Learning Internship
Week: Week 2
Version: 1.0
Date: August 15, 2026

Companion to the Business Understanding Report and
`week2_data_preprocessing.ipynb`.

Document Control

Programme	Machine Learning Internship
Week	Week 2
Project	Data Preprocessing & Feature Engineering for Machine Learning
Prepared by	Rudolf
Version	1.0
Dataset	WA_Fn-UseC_-Telco-Customer-Churn.csv (7,043 rows, 21 columns)
Output	telco_churn_processed.csv (7,043 rows, 27 columns)

Contents

1	Introduction	2
2	Part 1 – Data Inspection	2
3	Part 2 – Data Cleaning	2
3.1	Fixing the TotalCharges type	2
3.2	Handling the missing values	2
3.3	Duplicates	2
3.4	Cleaning up redundant categories	3
3.5	Dropping the identifier	3
4	Part 3 – Feature Engineering	3
5	Part 4 – Feature Encoding & Scaling	4
5.1	Encoding	4
5.2	Scaling	4
6	Part 5 – Outlier Detection & Feature Selection	5
6.1	Outliers	5
6.2	Correlation check	5
6.3	Variance threshold	5
6.4	Feature importance	5
7	Visualizations	5
8	Part 6 – Final Dataset	6
9	Summary	6
10	Conclusion	7

1 Introduction

This report covers the preprocessing pipeline built for Week 2 of the AnalystLab Africa internship. The Business Understanding Report sets out why ABC Communications needs a churn model in the first place; this document is about what happened to the data on the way to being model-ready, and why each step was taken rather than some alternative.

Everything described below happened in `week2_data_preprocessing.ipynb` and can be traced back to a specific cell there. Nothing here is asserted without having been checked against the actual dataset first.

2 Part 1 – Data Inspection

The raw file has 7,043 rows and 21 columns: a `customerID` identifier, 19 candidate predictors covering demographics, account details, subscribed services, and billing, and the `Churn` target.

Looking at `df.dtypes` turned up one real problem: `TotalCharges` is stored as an object (string) column even though it's a monetary amount. `SeniorCitizen` is stored as 0/1 rather than a category label, but that's not actually a problem – it's already in the form a model would want.

Running `df.isnull().sum()` reports zero missing values everywhere, which turns out to be wrong in one place. Because `TotalCharges` is text, its blank entries are empty strings, not `NaN`, so pandas doesn't flag them. The real missingness only shows up once the column is converted to numeric (Section 3.1).

Checked for both full-row duplicates and repeated `customerID` values – found neither. `df.describe()` was run across all columns to get a baseline read on distributions and category counts before touching anything.

3 Part 2 – Data Cleaning

3.1 Fixing the `TotalCharges` type

`TotalCharges` was converted with `pd.to_numeric(df["TotalCharges"], errors="coerce")`. This is the right call here because coercion turns every non-numeric entry into an explicit `NaN` instead of quietly mis-parsing it – and that's exactly what happened: 11 rows came back as missing.

3.2 Handling the missing values

Those 11 rows all share one thing: `tenure = 0`. These are customers who had just signed up – they have a `MonthlyCharges` figure on record but haven't been billed long enough to accumulate a `TotalCharges` total. So this isn't random missingness, it's a direct consequence of where these customers are in their billing cycle.

Given that, imputing with each customer's own `MonthlyCharges` made more sense than the two obvious alternatives. Dropping the rows would have thrown away exactly the new-customer segment that matters most for churn analysis, and filling with a dataset-wide mean or median would ignore information we already have about the customer (their actual monthly rate).

3.3 Duplicates

`df.drop_duplicates()` removed 0 rows, which lines up with what Part 1 already found. It's left in the pipeline anyway in case a future data refresh isn't as clean.

3.4 Cleaning up redundant categories

Six service columns (`OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`) use "No internet service" as a category alongside "Yes"/"No". That label is redundant – it only ever shows up when `InternetService == "No"`, so it's not telling us anything `InternetService` doesn't already say. Same situation with "No phone service" in `MultipleLines`, which duplicates `PhoneService == "No"`.

Both were collapsed into a plain "No". Keeping the longer labels would just have added extra one-hot columns later for no real gain.

3.5 Dropping the identifier

`customerID` was dropped straight away rather than waiting until feature selection to justify it statistically – a per-row unique ID has no predictive value by definition, and that was already established back in the business understanding phase.

4 Part 3 – Feature Engineering

Five features were added on top of the cleaned data, aimed at surfacing patterns that weren't directly visible in the raw columns.

AvgMonthlySpend

`TotalCharges / tenure`, rounded to two decimals (tenure treated as 1 where it's actually 0, to avoid dividing by zero). The idea was to capture spend rate as its own signal, separate from the raw totals. As it turns out, this didn't hold up – see Section 6.2.

TenureGroup

Tenure bucketed into four ordinal bands:

Group	Tenure range
0-1yr	up to 12 months
1-2yr	12–24 months
2-4yr	24–48 months
4yr+	over 48 months

The reasoning was that lifecycle stage (new vs. established customer) might matter more to a model than the exact month count, and bucketing makes that pattern easier to pick up in plots and in tree-based models specifically.

NumStreamingServices and NumSecurityServices

Two counts: streaming add-ons (`StreamingTV`, `StreamingMovies` – 0 to 2) and security/support add-ons (`OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport` – 0 to 4). The thinking here is that how many add-ons a customer has bundled might matter more than which specific ones, so collapsing four or two binary columns into a single count keeps that signal without the extra dimensionality.

HasInternet

A simple 0/1 flag from `InternetService` – 1 for DSL or Fiber optic, 0 for none. Mostly a convenience so models don't have to reconstruct this from the one-hot encoded version of `InternetService`.

Renaming

Nothing needed renaming – the original column names were already clear, and the new features were named to match the existing PascalCase convention.

Shape after this step: 7,043 rows, 25 columns.

5 Part 4 – Feature Encoding & Scaling

5.1 Encoding

Different columns needed different treatment depending on what kind of category they held:

Label encoding	gender, Partner, Dependents, PhoneService, MultipleLines, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, PaperlessBilling	Just two values each, so a single 0/1 column is all the information there is – no order gets implied that isn't there.
Ordinal encoding	Contract (Month-to-month < One year < Two year); TenureGroup (0-1yr < 1-2yr < 2-4yr < 4yr+)	These actually have a real order, and integer encoding keeps it. One-hot would have thrown that away.
One-hot encoding	InternetService, PaymentMethod	No natural order between categories like “Fiber optic” and “DSL”, so one-hot avoids inventing one.

Churn itself is mapped straight to 1/0 (Yes/No).

5.2 Scaling

`StandardScaler` was applied to `tenure`, `MonthlyCharges`, `TotalCharges`, and `AvgMonthlySpend`. These sit on very different scales – `tenure` tops out around 72, `MonthlyCharges` around 120, `TotalCharges` around 8,700 – and anything distance- or gradient-based (logistic regression, k-NN, SVM) would otherwise let `TotalCharges` dominate just because of its magnitude.

`StandardScaler` over `MinMaxScaler` because there's no real need to compress everything into $[0, 1]$, and doing so would flatten out relative differences that don't need flattening. `RobustScaler` wasn't necessary either – it exists for outlier-heavy data, and Part 5 below confirms there aren't meaningful outliers in these columns to protect against.

6 Part 5 – Outlier Detection & Feature Selection

6.1 Outliers

`tenure`, `MonthlyCharges`, and `TotalCharges` were checked three ways: box plots, the IQR rule ($1.5 \times \text{IQR}$ beyond $Q1/Q3$), and Z-scores ($|z| > 3$). All three agree – zero outliers, across all three columns.

That's not really surprising once you think about what these columns represent. Tenure can't exceed the observation window, and both charge columns are bounded by ABC's actual pricing tiers, so there's no room for the kind of extreme values IQR or Z-score would flag. No capping or winsorizing was applied, because there was nothing to apply it to.

6.2 Correlation check

A correlation heatmap was built across the numeric and engineered features. One result stood out: `AvgMonthlySpend` correlates with `MonthlyCharges` at roughly 0.996. Given how it was constructed – total charges divided by tenure, with the missing values imputed *from* `MonthlyCharges` in the first place – this makes sense in hindsight, but it wasn't obvious until the correlation was actually checked.

It was engineered as a reasonable hypothesis, but the data says it's basically a duplicate of `MonthlyCharges`, so it was dropped rather than kept around adding noise. This is really the point of doing feature selection with evidence instead of just keeping whatever you built.

6.3 Variance threshold

`Ran VarianceThreshold(threshold=0.01)` across all features except `Churn`. Nothing fell below it, so this step didn't remove anything beyond the `AvgMonthlySpend` drop above.

6.4 Feature importance

A quick Random Forest (200 trees, max depth 8, `random_state=42`) was fit on the encoded/scaled features (minus `AvgMonthlySpend`) against `Churn`, mainly to sanity-check the earlier analysis. `Contract`, `tenure`, `TotalCharges`, `MonthlyCharges`, `TenureGroup`, and `InternetService.Fiber optic` came out on top – which lines up with both the correlation results and the Week 1 framing that contract length and tenure are the main churn drivers here.

Everything else was kept for now. Nothing else fell below the variance threshold, and the importance ranking is more useful as a reference for later model-specific pruning than as a reason to cut anything else at this stage.

7 Visualizations

At least six charts were produced in the notebook, all titled and labeled:

1. Box plots of `tenure`, `MonthlyCharges`, `TotalCharges` – outlier check.
2. Correlation heatmap of the numeric and engineered features.
3. Feature importance bar chart (top 15, Random Forest).
4. Count plot of the `Churn` target.
5. Histograms with KDE for `tenure`, `MonthlyCharges`, `TotalCharges`.
6. Count plot of `Contract` type split by `Churn`.

7. Scatter plot of `tenure` vs. `TotalCharges`, colored by `Churn`.
8. Pair plot of `tenure`, `MonthlyCharges`, `TotalCharges` (1,500-row sample), colored by `Churn`.

8 Part 6 – Final Dataset

After dropping `AvgMonthlySpend`, the result was saved as `telco_churn_processed.csv`.

Rows	7,043
Columns	27 (26 features + <code>Churn</code>)
Missing values	0
Duplicate rows	0
Data types	fully numeric
Scaling	<code>StandardScaler</code> on the continuous columns

Fully numeric, no missing values, every category encoded according to what kind of category it actually is. Ready to split and hand off to modeling in Week 3.

9 Summary

Quick recap of what changed and why:

- `TotalCharges` converted to numeric, which exposed 11 hidden missing values.
- Those 11 rows (all `tenure` = 0) were filled with `MonthlyCharges`.
- No duplicates found, so nothing removed there.
- "No internet/phone service" labels collapsed into "No" across seven columns.
- `customerID` dropped.
- Five features added: `AvgMonthlySpend`, `TenureGroup`, `NumStreamingServices`, `NumSecurityService`, `HasInternet`.
- Encoding split three ways – label for binaries, ordinal for `Contract` and `TenureGroup`, one-hot for `InternetService` and `PaymentMethod`.
- `StandardScaler` applied to the continuous columns.
- No outliers found by IQR or Z-score, so no treatment applied.
- `AvgMonthlySpend` dropped after turning out to correlate at 0.996 with `MonthlyCharges`; variance thresholding removed nothing further; Random Forest importance confirmed `Contract` and `tenure/billing` features as the strongest predictors.
- Final dataset: 7,043 rows, 27 columns, saved as `telco_churn_processed.csv`.

10 Conclusion

Overall the dataset didn't need much heavy lifting – one type correction, one small and well-motivated imputation, no duplicates, and no numeric outliers to speak of. The bulk of the actual work here was in feature engineering and then being willing to walk one of those features back once the correlation analysis showed it wasn't adding anything. That's really the main lesson from this stage: build features based on reasonable hypotheses, but let the data decide which ones survive. The result is ready for Week 3.